Stall-Aware Residency for Graph ANNS on Unified CXL–SSD Memory

Anonymous Author(s)

Abstract

ACM Reference Format:
Anonymous Author(s). 2026. Stall-Aware Residency for Graph ANNS on Unified CXL–SSD Memory. In *Proceedings of the ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '27), March 2027, TBD*. ACM, New York, NY, USA, 5 pages.

1 Introduction

Approximate nearest-neighbor search (ANNS) has become a serving primitive for retrieval-augmented generation, recommendation, multimodal search, and online analytics. These services increasingly operate over corpora that are too large to be treated as private, per-server state. A graph ANNS index over billions of vectors can consume hundreds of gigabytes to terabytes once the vector payloads, graph links, quantization metadata, and auxiliary routing structures are taken into account. Replicating such an index across every compute host improves locality, but it also multiplies memory cost, slows index refreshes, and forces operators to provision peak memory for each independent replica. The natural alternative is one-copy disaggregation: keep a single shared corpus and index in a memory pool, then let many compute hosts serve queries concurrently against that shared state.

This paper studies the first one-copy disaggregated graph ANNS serving design that places the shared corpus on *flash-backed CXL–SSD memory*: a CXL-attached device that exposes memory semantics, satisfies missing pages through faults rather than explicit block I/O, stores cold capacity in flash-priced NAND, and has only a bounded amount of device DRAM for cached resident pages. Under this model, multiple decoupled hosts map the same index image and serve queries concurrently without maintaining private full copies. The contribution is not the label "CXL for ANNS" by itself. The new design point is the combination of shared-memory access, flash-scale economics, and a small contended cache sitting between irregular graph traversal and NAND.

This point is easy to blur because several neighboring designs appear similar at first glance. CXL memory-pooling systems show that memory-semantic disaggregation can simplify sharing, but they typically assume DRAM-class media, where a remote miss is expensive relative to local DRAM but still far from a flash miss. Distributed ANNS systems avoid per-host index duplication by partitioning or serving an index from memory servers, but their shared state again lives in DRAM-like memory or behind explicit RPC and storage interfaces. NVMe demand paging can run a large index from SSD capacity, yet it makes the SSD a storage-tier fallback for a single host rather than a shared memory-semantic substrate for many hosts. These baselines are valuable, but they miss the regime that motivates this work: a CXL-attached device whose capacity economics come from NAND and whose performance is governed by a small, contended device-DRAM residency set.

Flash-backed CXL–SSD memory changes the ANNS problem in three ways. First, the medium gap is large enough that a small increase in flash misses can erase the apparent cost benefit of moving from DRAM to NAND-backed capacity. A graph ANNS query is an irregular walk, not a scan: it repeatedly touches neighbor lists, vector codes, and candidate metadata selected by an evolving frontier. Second, the device cache is shared by all hosts. A policy that looks adequate for a warmed-up single host can collapse once multiple independent query streams compete for the same device DRAM. Third, ANNS exposes structure that generic page replacement misses. Pages on critical graph paths, hot entry regions, and frequently compared vector blocks are not equally valuable, and their value depends on how misses stall end-to-end search rather than on raw access counts alone.

We present a stall-aware residency system for graph ANNS on unified CXL–SSD memory. The system treats the CXL–SSD as the authoritative single copy of the index and coordinates host-side graph search with device-side residency decisions. Its key observation is that not all misses are equally harmful: a miss on a page that lies on many queries' critical search paths can amplify tail latency more than a miss on a cold vector payload or a rarely visited graph region. The system therefore makes residency a first-class ANNS decision. It lays out graph and vector pages for memory-semantic sharing, separates latency-critical search metadata from cold payloads, and uses search-time stall signals to bias the bounded device DRAM cache toward pages that reduce multi-host tail latency.

This paper makes the following contributions:

- We define flash-backed CXL–SSD memory as a distinct substrate for one-copy disaggregated ANNS: memory semantic like CXL, flash priced like SSD,

ASPLOS '27, TBD
2026.

and performance limited by a bounded device-DRAM cache.
- We design a multi-host graph ANNS serving architecture in which decoupled compute hosts concurrently query a shared, memory-semantic index image stored on NAND-backed CXL–SSD capacity.
- We identify three challenges unique to this point in the design space: the flash miss cliff, cross-host cache contention, and the need to translate ANNS search semantics into page residency decisions.
- We introduce stall-aware residency, which uses query execution stalls and page roles rather than access frequency alone to allocate scarce device-DRAM cache to the graph and vector pages that most affect latency.
- We evaluate the resulting one-copy serving design with placeholder experiments against replicated in-memory, DRAM-backed disaggregated, and SSD-demand-paging alternatives, targeting the conditions under which flash-backed CXL–SSD recovers memory cost without collapsing throughput or tail latency.

2 Background

2.1 Graph ANNS Serving

Graph-based ANNS indexes organize vectors as a proximity graph. A query starts from one or more entry points, repeatedly evaluates candidate neighbors, and expands the most promising frontier until a search budget is exhausted. The method is attractive for serving because it offers high recall with a tunable latency–accuracy trade-off, but its memory behavior is difficult for storage systems. Each query performs a data-dependent walk over small graph records, vector codes, and metadata. The next page to touch is determined by distances computed during the current query, so prefetching and batching opportunities are limited compared with scans or key-value lookups.

Production deployments therefore commonly keep graph indexes in host DRAM and replicate them across serving machines. Replication gives each host local, low-latency access, but it also turns index capacity into a per-host cost. A large corpus must be copied into every server that participates in serving, and updates must be propagated across those copies. One-copy serving removes this replication factor by placing the authoritative index image in a shared memory substrate and letting many compute hosts access it directly.

2.2 CXL Memory and Flash-Backed CXL–SSD

Compute Express Link (CXL) enables a host to map device-attached memory into its physical address space. A load or store to a mapped address can be served by a remote memory device without changing the application into an explicit block-I/O client. This memory-semantic interface is what makes CXL attractive for disaggregated indexes: existing pointer-rich or page-oriented data structures can be shared without rewriting the entire serving stack around RPCs.

Most CXL memory-pooling discussions assume DRAM-class capacity. In that regime, the main question is how much latency and bandwidth are lost when moving from local DRAM to a remote pool. A flash-backed CXL–SSD exposes a different point in the design space. It provides a CXL memory aperture, but the bulk capacity is NAND flash. A small amount of device DRAM caches resident pages; when a host touches a nonresident page, the device must fetch it from flash before the load can complete. The host sees a memory access that stalls, while the device sees a page residency problem under a strict cache budget.

2.3 Neighboring Baselines

This paper separates flash-backed CXL–SSD ANNS from three nearby baselines. First, replicated in-memory serving keeps all index state in each host's DRAM. It is fast but pays the full memory cost per host. Second, DRAM-backed CXL memory pooling and memory-server ANNS remove some duplication, but they still rely on DRAM-class shared capacity or explicit serving protocols. Third, NVMe demand paging places the index on SSD capacity and lets a host fault pages into memory, but the SSD remains a storage-tier backing device for that host rather than a memory-semantic, multi-host shared index. Flash-backed CXL–SSD combines properties from all three: the sharing goal of memory pooling, the capacity economics of SSDs, and the fault-driven execution model of demand paging.

3 Motivation

3.1 Challenge 1: The Flash Miss Cliff

The first challenge is the cliff between a CXL memory hit and a flash-backed miss. A naive argument for CXL–SSD ANNS is that NAND capacity is much cheaper than DRAM, so a single shared copy should dominate replicated in-memory indexes on cost. That argument is incomplete. Graph ANNS search is latency sensitive and pointer driven: a miss on a neighbor-list page can block the discovery of the next candidate, and a miss on a vector-code page can block distance evaluation for the current frontier. Once enough of these accesses fall out of device DRAM, the system pays a miss latency that can be tens to hundreds of times larger than a cache hit. The result is not graceful degradation; it is a throughput and tail-latency cliff.

Placeholder: miss-latency sensitivity for graph ANNS on flash-backed CXL–SSD. Plot throughput and p99 latency as device-DRAM hit ratio falls, highlighting the flash miss cliff.

Figure 1. Placeholder experiment for the CXL–SSD flash miss cliff.

Placeholder: multi-host cache contention. Compare one host, two hosts, and four hosts sharing the same device DRAM cache under identical aggregate and skewed query loads.

Figure 2. Placeholder experiment for cross-host contention in the CXL–SSD device cache.

Figure 1 will quantify this cliff. The expected comparison is not only CXL–DRAM versus CXL–SSD, but also CXL–SSD at different device-cache hit ratios. The key message is that flash backing is viable only if the residency policy keeps the search-critical working set out of the miss path.

3.2 Challenge 2: One Shared Cache for Many Hosts

The second challenge is cross-host cache contention. In one-copy serving, multiple compute hosts map the same index image, but they also compete for the same bounded device DRAM cache. A single host can make a query stream look warm by repeatedly touching nearby graph regions. When additional hosts arrive with different query distributions, those pages can be evicted by another host's frontier before reuse occurs. This effect is specific to the shared CXL–SSD setting: the device cache is neither a private host cache nor an infinite memory pool.

Figure 2 will show how contention changes both hit ratio and tail latency as the number of serving hosts increases. This motivates a residency policy that reasons about global utility across hosts rather than treating each host's page stream independently.

Placeholder: page role and stall contribution. Break down misses by graph metadata, neighbor lists, vector codes, and payload pages; report each class's contribution to end-to-end query stall time.

Figure 3. Placeholder experiment for translating ANNS search semantics into page residency decisions.

3.3 Challenge 3: ANNS Semantics Must Reach the Cache

The third challenge is that generic page replacement sees only addresses and recency, while graph ANNS exposes richer search semantics. A page containing upper-level entry points or high-fanout graph neighbors can influence many queries. A vector-code page may be cold in raw frequency but critical when it appears near the final reranking frontier. Conversely, some pages are touched often during exploratory search but have little impact on recall or tail latency. A frequency-only policy can therefore spend scarce device DRAM on the wrong pages.

Figure 3 will demonstrate why the cache should be guided by stall contribution, not by access frequency alone. The design goal is to connect what the host learns during graph search with what the CXL–SSD must decide when admitting, retaining, and evicting pages.

3.4 Challenge 4: Pipeline Bubbles Waste Flash Parallelism

Even when the working set is resident, a second inefficiency remains: a single graph query cannot keep enough reads in flight to exploit flash internal parallelism. Best-first search is pointer driven—the next hop is unknown until the current node's page is fetched and scored—so a demand-paged query collapses to a device queue depth near one. Pipelined search (e.g., PipeANN) relaxes the strict compute–I/O order within a query, but the pipe still bubbles: it starves when the candidate pool has no unread neighbor to issue, its dynamic width ramps slowly, and it drains at query end. Figure 4 illustrates these bubbles and how filling them requires *cross-query* work. On a Dell PM1733a NVMe with a PipeANN disk index over 984,133 1024-d embeddings, a single-query pipeline reaches only an average device queue depth of 9.8 (utilization 0.53–0.92; 14–42% of loop iterations starve). Interleaving a static batch of independent queries

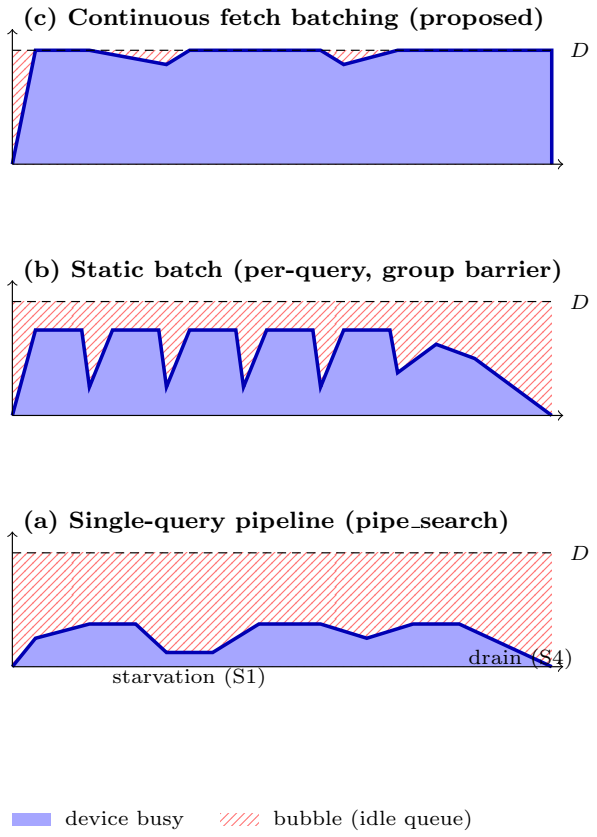

Figure 4. Pipeline bubbles under three schedulers (schematic). The y-axis is in-flight device reads; D is the target queue depth needed to saturate flash internal parallelism. (a) A single query is pointer-chasing, so it cannot fill the pipe (measured device queue depth aqu-sz ≈ 9.8, utilization 0.53–0.92, 14–42% of loop iterations starved). (b) Interleaving a static batch of independent queries raises occupancy (aqu-sz ≈ 13.8, +44% QPS) but a per-hop/group barrier drains the tail. (c) Continuous fetch-level batching keeps the device near D (aqu-sz ≈ 14.4 single core; 319 at 16 threads) at identical recall (86.5%). Single-core numbers: 984,133×1024-d embeddings, PipeANN disk index, Dell PM1733a NVMe.

raises this to 13.8 (+44% QPS), and continuous fetch-level batching keeps the device fullest—at identical recall. The remaining gap to full saturation widens exactly on flash-backed media, where each avoidable idle slot is a missed opportunity to hide a miss that costs orders of magnitude more than a hit.

4 Design

4.1 Overview

Our design turns the flash-backed CXL–SSD into the authoritative memory image for a graph ANNS index.

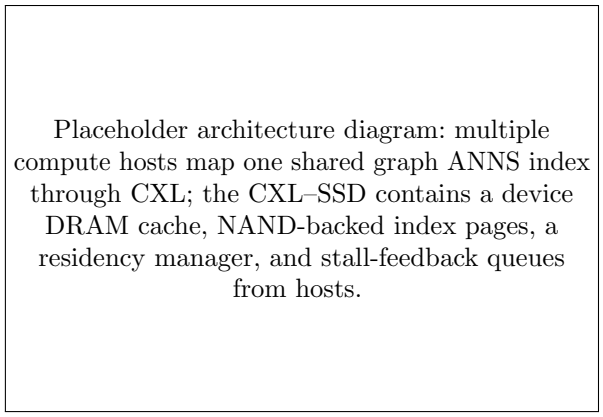

Figure 5. Placeholder architecture of one-copy graph ANNS serving on flash-backed CXL–SSD memory.

Compute hosts remain responsible for query execution: they maintain per-query frontiers, compute distances, and decide which graph neighbors to expand. The CXL–SSD is responsible for serving memory-semantic page accesses from a two-tier device hierarchy: a bounded DRAM cache backed by NAND flash. The central design question is how to keep the pages that matter to search progress resident in device DRAM while many hosts concurrently traverse the same index.

Figure 5 summarizes the architecture. The design has four components. First, a shared index image gives every host the same virtual view of graph and vector pages. Second, a page-aware layout separates latency-critical graph metadata from colder vector payloads. Third, a stall-aware residency manager ranks pages by their effect on query stalls under the current multi-host workload. Fourth, a lightweight feedback path lets hosts report search context without moving query execution into the device.

4.2 D0: One Shared Memory Image

The index is stored once on the CXL–SSD and mapped by all serving hosts. The shared image contains the graph topology, vector encodings, entry-point metadata, and any auxiliary search tables needed by the ANNS implementation. Hosts issue normal memory accesses against this image; when a page is resident in device DRAM, the CXL–SSD returns it as a memory hit, and when it is not resident, the device faults the page in from NAND before completing the access.

This design deliberately avoids two alternatives. It does not replicate the full index in host DRAM, because that would lose the one-copy property. It also does not convert the index into a remote search service, because that would hide the memory behavior behind RPC and prevent the host search loop from reacting

to page-level stalls. Keeping the index memory semantic lets the system reuse graph ANNS execution while exposing residency as the primary resource to manage.

4.3 D1: Page-Aware Index Layout

The layout groups index objects according to their role in search. Entry points, upper-level graph metadata, and high-reuse neighbor lists are separated from bulk vector payloads and low-reuse graph regions. This separation gives the residency manager meaningful admission units: keeping a page resident can be interpreted as keeping a specific type of search state close to the hosts, rather than as caching an opaque block of bytes.

The layout also preserves one-copy sharing. All hosts see the same page image, so a page admitted for one host can benefit another host that soon traverses the same graph region. The challenge is to make this sharing constructive rather than destructive: page groups should align with ANNS reuse patterns so that multi-host activity increases useful sharing instead of only increasing cache churn.

4.4 D2: Stall-Aware Residency

The residency manager uses stall contribution as the primary signal for admission and eviction. For each page, the system observes not only whether the page was touched, but where the touch occurred in the search loop and how much it delayed query progress. A miss that blocks frontier expansion, candidate reranking, or a frequently reused graph entry receives higher priority than a miss on a cold payload page. This makes the policy different from recency-only or frequency-only replacement: the unit of value is saved query stall time.

The policy maintains a small set of page-role counters and stall scores. Hosts attach lightweight context to faulting accesses, such as whether the page holds graph neighbors, vector codes, or reranking data. The device combines this context with observed residency outcomes to decide whether a faulted page should enter device DRAM, whether an existing page should be retained, and which page should be evicted when the cache is full.

4.5 D3: Multi-Host Coordination

In a multi-host deployment, the same page can have different utility across query streams. The design therefore aggregates stall signals at the CXL–SSD instead of letting each host independently decide what is important. A page that reduces tail latency for several hosts should survive short-term pressure from a single bursty stream, while a page that is repeatedly touched by one host but produces little stall reduction should not monopolize the shared cache.

The coordination mechanism is intentionally narrow. Hosts do not send full queries, graph frontiers, or vector contents to the device. They send enough metadata to classify the stalled access and attribute it to a query phase. This keeps query execution on the compute hosts while giving the CXL–SSD the one piece of semantic information generic demand paging lacks: which resident pages actually shorten ANNS search under shared-cache contention.

5 Evaluation

6 Related Work

7 Conclusion

References